California State Polytechnic University Pomona
Department of Electrical and Computer Engineering
ECE 3709L FINAL Project
Wireless PID Motor Control with NRF24L01+, I²C LCD, Sensors & LabVIEW

1. Overview – How Lab 9 Extends Lab 8

In Lab 8, you designed a complete wireless DC motor controller using:

- Arduino Uno (receiver)
- NRF24L01+ (2.4 GHz wireless)
- L293D motor driver
- LabVIEW GUI that sent Forward / Backward / Stop commands
- I²C LCD to display status
- LEDs for dimmer indication
- 5 V power architecture
- Students wrote all motor control code themselves

What's New in Lab 9 (Added Features)

Lab 9 builds directly on Lab 8, but adds a closed-loop feedback system using PID Control.

New requirements:

A. Add a PID Control System

Students must:

- Measure motor performance (RPM estimate OR feedback variable)
- Design a PID controller (P, I, D terms)
- Tune the PID loop
- Use Arduino to compute:

$$PWM_{output} = K_p e(t) + K_i \int e(t)dt + K_d \frac{de(t)}{dt}$$

- Control DC motor speed wirelessly through LabVIEW

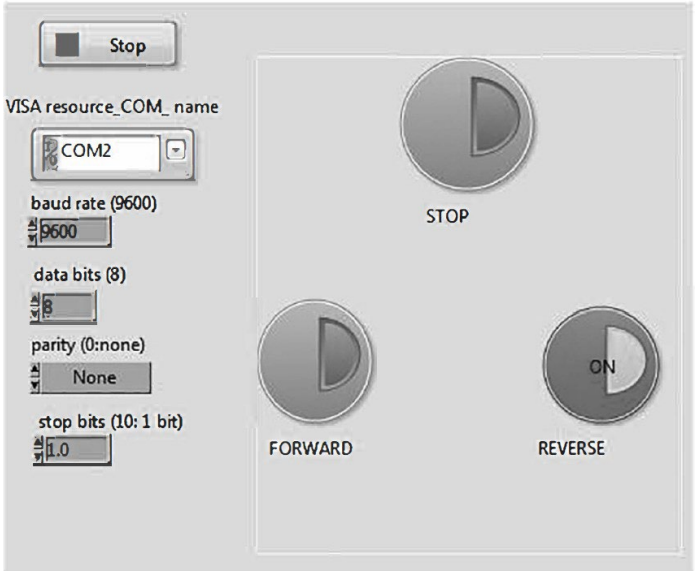
B. Add Sensors for Feedback & Monitoring

You must integrate these sensors into the system:

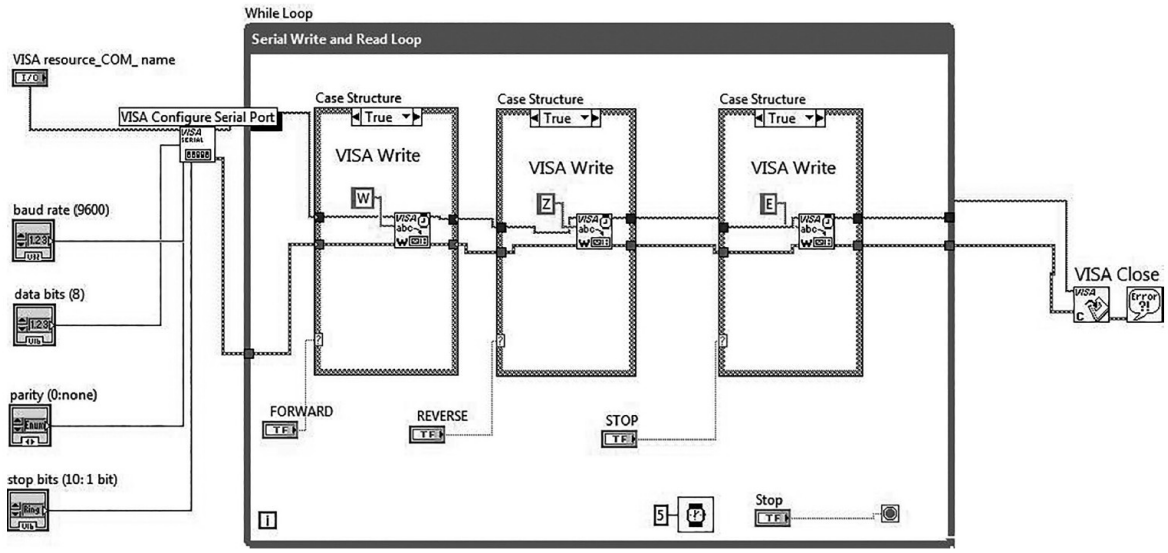
- TC74A0-3.3V temperature sensor (I^2C)
- DHT11 humidity & temperature
- LDR for ambient light
- Optional: 28BYJ-48 stepper motor (as secondary load or for students to demonstrate multi-motor control)
- Add capacitor + inductor for power stability (students choose values)

C. Modify the LabVIEW GUI

Below is the LabView GUI you designed in lab 8



Front panel showing that the motor is controlled in "FORWARD" direction



LabVIEW GUI Block diagram

The Lab 9 GUI is the extended version of the same design:

- Add Setpoint (RPM or PWM%)
- Add Start PID
- Add Display of actual motor feedback
- Display temperature/humidity/light data

D. System-Level Additions

You will:

- Add pushbuttons for local Start/Stop/PID enable
- Update LCD to show:
    - Mode: Manual or PID
    - Setpoint
    - Actual speed
    - Temperature
    - System status

2. Learning Objectives

By the end of Lab 9, you will be able to:

1. Integrate multiple sensors using I^2C and ADC inputs
2. Implement wireless command/control using NRF24L01+
3. Design and tune a PID controller
4. Use LabVIEW for real-time closed-loop control
5. Control DC motors and stepper motors from a single system
6. Understand hierarchical system integration (Motor + Wireless + Sensing + GUI)

3. Required Hardware

Microcontroller & Communication

- Arduino Uno (main controller)
- NRF24L01+ Wireless Transceiver (with 3.3V regulator module)

Motor & Drivers

- DC Motor (3V–12V, 25,000 RPM)
- L293D Motor Driver (H-Bridge)
- 28BYJ-48 Stepper Motor + ULN2003 Driver

Input/Output

- I^2C LCD 1602 Display
- 4 LEDs (dimmer level)
- 3 push buttons (Start / Stop / PID Enable)

Sensors

- TC74A0-3.3VAT Temperature Sensor (I^2C)
- DHT11 Temperature + Humidity
- LDR + 10 kΩ resistor (voltage divider)

Power

- 5 V supply for Arduino & logic

- **Separate 5–12 V supply for the DC motor**
- Capacitor of choice (students must justify)
- Inductor of choice (students must justify)

(Any capacitor and inductor can be used; you must explain their choice.)

4. System Architecture

4.1 Power System

- Arduino Uno + NRF24L01+ + LCD + sensors run from 5 V USB or 5 V regulator
- DC motor powered by separate supply (3–12 V)
- Grounds must be connected together

4.2 Wireless System

- LabVIEW → Arduino Transmitter → Wireless packet → Arduino Receiver → Motor system
- Commands: Forward, Backward, Stop, Setpoint, PID Enable

5. PID Control Requirements

5.1 PID Structure

Let:

- **Setpoint** = desired motor speed (RPM or PWM%)
- **PV (process variable)** = measured feedback (motor speed or back EMF estimation)
- **e(t)** = setpoint − PV

You write code for:

- Error calculation
- Proportional term
- Integral accumulation
- Derivative calculation

5.2 Sampling Frequency

You must:

- Choose a loop interval (e.g., 10 ms)
- Explain why the sample period matters

5.3 Feedback Measurement Options

You choose one:

1. RPM estimation using interrupt on motor encoder (if available)
2. Back-EMF measurement using analog input
3. Current sensing (optional)
4. Approximate feedback using PWM back-calculation (allowed if motor has no encoder)

5.4 PID Output

PID output controls:

- L293D → PWM → motor

6. What You Must Implement in Code

What You need to implement:

A. Wireless command receiving (NRF24L01+)

- Parse commands from LabVIEW
- Example pseudo-comments:
- // Read struct from RF
- // If command == 'PID', switch mode
- // If command includes setpoint, update setpoint variable

B. PID loop

// error = setpoint - measured_speed

// P = Kp * error

// integral += error * dt

// I = Ki * integral

// derivative = (error - previous_error) / dt

// D = Kd * derivative

// output = P + I + D

// Constrain output between 0–255

C. Motor Control

// Use PWM on enable pin of L293D

// Direction determined by forward/backward command

D. LCD Display

- Mode (Manual / PID)
- Setpoint
- Measured speed
- Temperature (TC74A0)
- Humidity (DHT11)
- Light level (LDR)

E. Sensor Acquisition

// Read TC74 using I2C

// Read DHT11 digital values

// Read LDR value using analogRead

F. Push Buttons

- Local override:
    - Start
    - Stop
    - PID enable/disable

G. Stepper Motor (Secondary Task)

Students must control the stepper motor using ULN2003:

- Stepper may move based on ambient light or temperature
- This demonstrates multi-motor scheduling

7. LabVIEW Requirements

You extend the GUI of lab 8:

New GUI Controls

- Setpoint input
- PID enable button
- Mode indicator
- Motor direction buttons

New GUI Indicators

- Motor speed (feedback)
- Motor temperature
- Ambient temperature & humidity
- Light level
- System status text

8. Wiring Overview, you need to draw the final diagram in your lab report.

8.1 Arduino → L293D

- PWM → EN1
- Pin X → IN1
- Pin Y → IN2
- Motor → OUT1/OUT2
- Motor Vcc → External supply
- Grounds tied together

8.2 I^2C LCD

- SDA → A4
- SCL → A5

- Vcc → 5 V
- GND → GND

8.3 TC74A0 (3.3V)

- Connect to **3.3V** pin of Arduino
- SDA → A4
- SCL → A5

8.4 DHT11

- Data → Arduino digital pin
- 5 V power
- Use 10 k pull-up

8.5 LDR

- LDR ⇔ 5 V
- LDR–10k divider → A0

8.6 NRF24L01+

- Requires **3.3V**
- CE → D9
- CSN → D10
- SCK → D13
- MOSI → D11
- MISO → D12

8.7 Stepper + ULN2003

- IN1–IN4 → Arduino pins
- Powered from 5 V

IR Optocoupler for Motor Speed Detection:

measuring DC motor speed (RPM) is the hardest part of this project, because your 3–12 V 25,000 RPM motor does not come with an encoder, and small toy DC motors don't make speed sensing easy. You need to by LM393 from Amazon, it's 2$. This is the most reliable, easiest to mount, Arduino-friendly, and accurate way to measure RPM for a 3–12 V, 25,000 RPM DC motor. It is the method that almost every university lab, robotics kit, and industrial teaching platform uses.

Your motor can reach 25,000 RPM.
A slotted encoder with 20–40 slots handles this with no problem.

Example:

```
25,000 RPM × 20 slots = 500,000 pulses/min = 8,333 pulses/sec (8.3 kHz)
```

Arduino Uno digital interrupt pins can easily read 8 kHz pulses.

The interrupter gives square wave pulses, not analog noise.

Arduino can measure pulses using:

- `attachInterrupt()`
- Micros-based timing
- Pulse counting every fixed time window

Once you have clean RPM, PID is straightforward:

```
error = rpm_setpoint – rpm_measured
```

This is the classic control systems example.

- Photo Interrupter Module
- Speed Measurement Module
- LM393 Optical Encoder Module

Output:

- 5V logic square wave
- Works directly with Arduino Uno digital pins (no level shifter required)

How you mount it

1. Put the small encoder disk on:
    - The motor shaft
      or

    - A small shaft/flexible coupling attached to the motor
2. Insert the disk into the U-shaped interrupter.
3. Each slot → 1 pulse.

How Arduino measures RPM (simple math)

If the disk has **N slots**:

Count pulses for 100 ms

```
rpm = (pulse_count × 600) / N
```

or measure time between pulses

```
rpm = 60,000,000 / (period_in_microseconds × N)
```

This is simple for students to implement.

9. What You Must Submit:

A. Complete PID Analysis

1. Explanation of chosen Kp, Ki, Kd
2. Step response plot (before and after tuning)
3. Discussion of overshoot, settling time, stability

B. Full Block Diagram

- Wireless → PID → Motor → Sensors → GUI

C. Final Wiring Schematic

D. Full Arduino Code into Canvas

- Must include comments for PID, wireless, LCD, sensors

E. LabVIEW Front Panel + Block Diagram screenshots

F. In person Demonstration

- GUI setpoint changes motor speed
- PID smooths the response
- Sensor readings on LCD & GUI